A MACHINE-VERIFIED FAMILY OF ANTI-RECURRENCE SEQUENCES: THE RUSTY NUMBERS AND 248 FURTHER LINEAR FORMS

ANDREW HINGSTON

Abstract. Fokkink and Joshi's anti-recurrence conjecture (arXiv:2506.13337; published as *Anti-recurrence sequences*, Integers **26** (2026), #A24, Conjecture 2) predicts that for every positive integral linear form $a = (a_1, \ldots, a_k)$ of dimension $k > 1$, the anti-recurrence sequence A_n satisfies $A_n - \kappa n$ τ-automatic, $\kappa = k\tau + 1$, τ the trace. The authors prove this, their Theorem 4, only under an "A_1-boundedness" restriction, and single out the *rusty numbers* $a = (1, d)$ as a case where, in their words, "proving or disproving the conjecture ... remains a challenge" — every $d \geq 3$ falls outside A_1-boundedness. We report a machine *verification*, not a proof, of Conjecture 2 for 249 positive integral linear forms, including the rusty numbers $a = (1, d)$ for $d = 3, \ldots, 12$: of the 249, 61 — including every one of these rusty numbers — are not A_1-bounded and so are reached by no published theorem, while the conjecture in general is not addressed and remains open. Each verification guesses a candidate DFAO for the difference sequence and discharges a fixed list of closed first-order sentences over $\langle \mathbb{N}, +, V_\tau \rangle$ plus the DFAO through a decision procedure (Peanut) — $k + 9$ of them, eleven for the rusty numbers, where $k = 2$; an induction on n turns these `TRUE` sentences into a proof that the encoded sequence *is* the anti-recurrence sequence. One predicate in the first write-up of this method, `LAST`, was inverted, silently making two of the obligations vacuous; this was caught in refereeing, the predicate was corrected, and all 249 records were independently re-verified with the fix. For $a = (1, 4)$ we exhibit an explicit 9-state base-5 DFAO and show it explains, rather than merely reproduces, the anomaly the published paper reports by hand ($A_{5n+1} = 9 + 55n$ first fails at $n = 1741$): the automaton has exactly one state at which the recursive step deviates, first reached at $n = 348$. Two near-miss automata are exhibited that reproduce the correct sequence for 13 535 104 and 51 280 580 terms respectively and are still wrong, each refuted by an engine-produced counterexample rather than by longer prefix search — offered as evidence that the method's positive verifications are trustworthy. Across the 249 forms the minimal state count of $a = (1, d)$ obeys $|Q|(d + 3) = |Q|(d) + 10$ for every $d \geq 3$, an empirical law used to predict, and then confirm, $|Q|(11)$ and $|Q|(12)$ before those two forms were run.

1. Background and the conjecture

For strictly increasing sequences of positive integers A_n, B_n partitioning $\mathbb{N}_{>0}$, and a positive integral linear form $a = (a_1, \ldots, a_k)$, $k > 1$, of *trace* $\tau = \sum_j a_j$, the *anti-recurrence sequence* generated by a is defined by

$$A_n \;=\; \sum_{j=1}^{k} a_j \, B_{(n-1)k+j},$$

where the n-th *B-block* $B_{(n-1)k+1}, \ldots, B_{(n-1)k+k}$ is the k smallest positive integers not yet placed in $A \cup B$ (Fokkink–Joshi's Lemma 3 shows such sequences exist and are unique, generated blockwise by this mex rule). The anti-Fibonacci numbers, $a = (1, 1)$, are the case popularised by Hofstadter and Zaslavsky [5]; Bosma et al. [2] name the general phenomenon the Clergyman's Conjecture and settle $a = (1, 1, 1)$ and $a = (1, 1, 1, 1)$. Kimberling and Moses [4] formulated several questions about such sequences that together suggest the general meta-conjecture; Fokkink and Joshi [3] give the precise statement:

Conjecture 1.1 ([3, Conj. 2])**.** *Let $a = (a_1, \ldots, a_k)$ be positive and integral of dimension $k > 1$. Let A_n be the anti-recurrence sequence generated by the linear form $f(x) = a \cdot x$. Then $A_n - \kappa n$ is τ-automatic for $\kappa = k\tau + 1$ and τ the trace of the linear form.*

Definition 1.2 ([3, Def. 1])**.** A positive linear form a of dimension k and trace τ is A_1-*bounded* if $A_1 \leq (k-1)\tau + 2$, where A_1 is the first anti-recurrence number in the sequence generated by a.

> **Theorem 4** (Fokkink–Joshi [3]). *If a is A_1-bounded, then it generates an anti-recurrence sequence A_n such that $A_n - \kappa n$ is τ-automatic.*

The proof of Theorem 4 is constructive: it produces an explicit DFAO recognising the difference sequence, with at most $2\tau - 1$ states.

Date: 18 August 2026.
2020 *Mathematics Subject Classification.* 68Q45, 68R15, 11B85.
Key words and phrases. anti-recurrence sequence, automatic sequence, DFAO, decision procedure, mex.

Section 5 of [3] (arXiv §4) introduces $a = (1, d)$, "the rusty numbers," exhibits a guessed 4-state DFAO for $d = 3$, reports that for $d = 4$ the subsequence A_{5n+1} agrees with the arithmetic progression $9 + 55n$ only up to $n = 348$ ("when $A_{1741} \neq 9 + 55 \cdot 348$"), and concludes:

> "Surprisingly, proving or disproving the conjecture for the rusty numbers remains a challenge." [3, §5]

Its closing section asks, verbatim:

> "Does the conjecture hold without the restriction of A_1-boundedness? Are the rusty numbers sums of linear sequences and automatic sequences?" [3, §6]

For $a = (1, d)$, $k = 2$, so $A_1 = \sum_j ja_j = 1 + 2d$ and $\tau = 1 + d$, giving $\kappa = 2\tau + 1 = 2d + 3$; A_1-boundedness reads $1 + 2d \leq (k-1)\tau + 2 = (1 + d) + 2 = d + 3$, i.e. $d \leq 2$. So $(1, 1)$ and $(1, 2)$ are covered by Theorem 4 (also, respectively, by Zaslavsky [5] directly and by Fokkink–Joshi's Theorem 1 for the anti-Pell numbers), and **every rusty number** $d \geq 3$ **falls outside every published proof of Conjecture 1.1** — exactly the case the authors call a challenge.

2. What is established here

We report machine *verifications* of Conjecture 1.1 for 249 positive integral linear forms a, chosen by a sweep over dimension $k = 2, \ldots, 4$ and bounded component size, including every rusty number $a = (1, d)$, $d = 1, \ldots, 12$. Of these, 61 forms — including $a = (1, d)$ for $d = 3, \ldots, 12$ — are *not* A_1-bounded and so are not covered by any theorem in [3]. ($a = (1, 3)$ is a partial exception: [3] exhibits a 4-state DFAO for it in Figure 7 without a transcript or state count; we independently verify a minimal 6-state automaton.)

Each of the 249 verifications consists of: (i) a candidate DFAO for the difference sequence $A_n - \kappa n$, produced by guessing the Myhill–Nerode quotient of a computed prefix of the sequence; (ii) a fixed list of $k+9$ closed first-order sentences over Presburger arithmetic with a base-τ numeration predicate (11 for the dimension-two forms, including every rusty number), built from the candidate DFAO, each checked `TRUE` by the decision procedure Peanut; and (iii) a hand induction, §3 below, showing that `TRUE` on those sentences implies the candidate DFAO encodes exactly the mex-generated pair (A_n, B_n). Steps (ii)–(iii) are what make this a proof for the specific a verified, rather than a numerical coincidence: unlike prefix matching, a `TRUE` first-order sentence over $\langle \mathbb{N}, +, V_\tau \rangle$ decided by the Bruyère–Hansel–Michaux–Villemaire theorem [1] is a statement about *all* $n \in \mathbb{N}$, not a finite check.

We want to be exact about what this is and is not. **It is not a proof of Conjecture 1.1.** What is proved is a large finite list of instances of the conjecture, one a at a time; nothing here is uniform in a, no infinite subfamily is closed, and the general case — including whether *every* $a = (1, d)$ works, not just $d \leq 12$ — is exactly as open after this note as before it. What *is* new is that the specific challenge [3] names, the rusty numbers, is machine-verified for the first ten values past the A_1-bounded range, each with an explicit minimal DFAO and a complete, checked induction.

3. Method: encoding, the corrected predicate, and the induction

Automatic sequences are 0-indexed; write $A(m) := A_{m+1}$, $m \geq 0$. Fix a base-τ DFAO T (constants lo, hi; output alphabet $\{0, \ldots, \mathrm{hi} - \mathrm{lo}\}$) and put

$$AA(m, s) \;:\equiv\; s = \kappa m + \mathrm{lo} + T[m] \qquad (\text{“}A_{m+1} = s\text{”}),$$

a finite disjunction over output letters. Write the n-th B-block as $x_j = x + (j-1) + [j > g]$, $j = 1, \ldots, k$, for some $g \in \{1, \ldots, k\}$ ($g = k$ meaning no skip, otherwise the value $x + g$ is skipped); then $A(m) = \tau x + c_g$ with $c_g = (A_1 - \tau) + \sum_{j>g} a_j$. Since $0 \leq \sum_{j>g} a_j \leq \tau - a_1 < \tau$ and this sum is strictly decreasing in g, the c_g are pairwise distinct mod τ, so (x, g) is recovered from $A(m)$ with no existential quantifier:

$$\begin{aligned}
G_g(m, x) &:\equiv AA(m, \tau x + c_g) && \text{(one \texttt{let} per } g\text{)}\\
B1(m, x) &:\equiv \bigvee_g G_g(m, x) && \text{block start}\\
\mathit{LAST}(m, y) &:\equiv \bigvee_{g<k} \big(\exists x.\, G_g(m, x) \wedge y = x + k\big) \;\vee\; \big(\exists x.\, G_k(m, x) \wedge y = x + k - 1\big) && \text{block end}\\
\mathit{INBLK}(m, t) &:\equiv \bigvee_{g,j} \exists u.\, G_g(m, u) \wedge t = u + (j-1) + [j > g]\\
\mathit{USED}(j, t) &:\equiv AA(j, t) \vee \mathit{INBLK}(j, t)\\
\mathit{ISA}(t) &:\equiv \exists n.\, AA(n, t) \qquad \mathit{ISB}(t) :\equiv \exists m.\, \mathit{INBLK}(m, t)
\end{aligned}$$

Ten fixed closed sentences (functionality/totality of $AA/B1$; the base case $G_k(0,1) \wedge AA(0,A_1)$; monotonicity; $LAST$ strictly below the next block; complementarity of A and B; every value below a block already used) plus one instance of "the skipped value already used" per skip-position $g = 1,\ldots,k-1$ — $k+9$ sentences in all, eleven when $k=2$, as for every rusty number — are then checked `TRUE`. Standard induction on m — using $LAST$ to place the new block strictly above every earlier B-value, complementarity to place it strictly above every earlier A-value, and the "used below x" obligation to force $x = \mathrm{mex}$ — shows the encoded pair equals the mex-generated (A_n, B_n) term by term, hence $A_n - \kappa n = (\mathrm{lo} - \kappa) + T[n-1]$ is τ-automatic: Conjecture 1.1 for that a.

The bug, and the fix. The first write-up of this method, and the script implementing it, defined $LAST$ by substituting $x := y + k$ into G_g, i.e. $LAST(m,y) :\equiv \bigvee_{g<k} G_g(m, y+k) \vee G_k(m, y+k-1)$. That asserts $y = x - k$, the block start *minus* k, not the block's last element $x+k-1$ or $x+k$ — the wrong direction. The bug made two of the (fixed, k-independent) obligations built from $LAST$ vacuously true for every candidate DFAO, silently dropping the step of the induction that uses $LAST$ to place the new block above the previous one; every other obligation, and the overall `TRUE`/`FALSE` verdict, was unaffected in the sense that no false candidate happened to pass, but the argument as written did not establish what it claimed to. This was caught in refereeing (`research/referee-verdicts/attack3-verdict.md`, an independent, adversarial re-derivation and re-implementation of every step from scratch), which also confirmed by direct construction that with the inverted predicate the obligation fires on 0 of 560 hand-perturbed near-miss automata, i.e. it was carrying no weight. With the one-line fix above, all 249 records were rebuilt with an independently written script and re-run: 249/249 all obligations `TRUE`, 0 failures, total engine time 541.2 s, worst case 14.9 s ($a = (1,12)$, 36 states). The conclusion — every one of the 249 forms verified, including the rusty numbers $d = 3,\ldots,12$ — is unchanged by the fix; what changes is that the argument now actually supports it.

4. The named case: $a = (1,4)$

Here $\tau = 5$, $\kappa = 11$, $A_1 = 9$; A_1-boundedness requires $A_1 \le 7$, so Theorem 4 does not apply. A 9-state base-5 DFAO for the difference sequence was guessed and verified (all eleven obligations `TRUE` in 0.1 s), giving

$$A_{m+1} \;=\; 11m + 5 + T[m].$$

This explains the anomaly [3] reports by hand. Exactly one state q_8 of the nine has $\delta(q_8, 0) \ne q_0$ (instead $\delta(q_8,0) = q_1$); every other state maps 0 back to q_0. Since $\mathrm{state}(5n) = \delta(\mathrm{state}(n), 0)$, the subsequence A_{5n+1} is an arithmetic progression exactly while $\mathrm{state}(n) \ne q_8$, and the first n whose base-5 expansion reaches q_8 is $n = 348$ (base-5 expansion 2343). Concretely $A_{1741} = 19148 = 11\cdot 1740 + 5 + T[1740]$, one less than $9 + 55\cdot 348 = 19149$: the automaton reproduces [3]'s reported anomaly to the exact index, and explains it as a single transition, not a numerical accident.

Negative controls. Two under-sized guesses illustrate why the eleven obligations, and not prefix agreement, are what makes a verification trustworthy. A 35-state candidate for $a = (1,12)$ reproduces the true mex-generated sequence for 51,280,580 consecutive terms and is still wrong: the complementarity obligation and the "every value below a block already used" obligation both return `FALSE`. Rather than search further by hand, we ask Peanut for a witness to the negated obligation directly (`witness $B1(m,x) & t>=1 & t<x & ~(E j.  j<m & $USED(j,t))`); it returns the exact breaking index $m = 51{,}280{,}580$ in seconds. Re-guessing from a prefix past that index gives a genuine 36-state DFAO, which then verifies. A second example, $a = (2,9)$: a 26-state candidate reproduces the mex definition for the 1.2×10^7 terms originally computed, and the engine's witness locates its actual first failure at $m = 13{,}535{,}104$; re-guessing past that index gives the correct 27-state DFAO. In both cases the engine is used as an equivalence oracle against the mex generator's membership oracle, so the length of prefix needed is discovered rather than guessed.

5. Verification

Independently of the guess-and-verify pipeline above:

- A second, separately written mex generator (a naive `mex`-over-a-`set`, no pointer arithmetic) agrees with the generator used to produce the 249 candidates on A and B for the first 3000 terms of 13 forms spanning $k = 2,3,4$.
- All 249 stored DFAOs were rebuilt from their committed `def` (morphism/coding) strings and replayed against a fresh run of the naive generator for $n \le 20{,}000$: 0 mismatches, on a code path sharing nothing with the guesser.
- The generator reproduces published values exactly: OEIS A075326 ($a = (1,1)$), A304502 ($a = (1,2)$), A304499 ($a = (2,1)$), A265389 ($a = (1,1,1)$), A299409 ($a = (1,1,1,1)$), and both published rusty lists, $a = (1,3)$ and $a = (1,4)$, from [3, §5].

- An independent referee re-implementation (`paper/verdict-attack3/`) built its own obligation-generating script from scratch, never importing the code above, and its own generators (a pure-Python `mex` over a `set`, and a bitset generator in C); it re-ran all 249 records with the corrected `LAST` and obtained 249/249 `TRUE`, and regenerated the $(1, 12)$ sequence independently to 7×10^7 terms with 0 mismatches against the stored 36-state DFAO.

6. Observations

Two structural patterns were noticed across the 249 verified forms; both are reported as **observations on the data**, not as theorems — neither is proved here, and neither follows from anything in §3.

Observation 6.1. Across all 249 forms the difference sequence's output alphabet has width $\mathrm{hi} - \mathrm{lo} + 1 \leq 2\tau - 2$, i.e. one less than the $2\tau - 1$ that Theorem 4 proves under A_1-boundedness; this persists even for the 61 forms not covered by Theorem 4.

Observation 6.2 (state-count law for the rusty numbers)**.** The minimal base-τ DFAO of $a = (1, d)$ has, for $d = 1, \ldots, 12$,

$$|Q|(d) \;=\; 2,\ 3,\ 6,\ 9,\ 11,\ 16,\ 19,\ 21,\ 26,\ 29,\ 31,\ 36,$$

and $|Q|(d + 3) = |Q|(d) + 10$ for every $d \geq 3$ (7 instances, $d = 3, \ldots, 9$). The law fails at $d = 1, 2$ (the two A_1-bounded members: $+7$ and $+8$). **Two of the seven confirming instances were predictions made before the corresponding DFAO was computed**: the law, fit to $d \leq 9$, predicted $|Q|(11) = 31$ and $|Q|(12) = 36$; both were then run and confirmed, the second only after Peanut's counterexample killed the 35-state near-miss of §4. Every value is the Myhill–Nerode quotient of a machine-verified automaton, hence an exact minimal count, not an estimate. The law's own extrapolation gives the untested predictions $|Q|(13) = 39$, $|Q|(14) = 41$, $|Q|(15) = 46$.

7. Limits

Conjecture 1.1 is not proved. What is proved is a finite — if unusually large, and individually non-trivial — list of instances. No argument here is uniform in a; Observation 6.2 says a general proof cannot simply relax the constant $2\tau - 1$ of Theorem 4, since the minimal alphabet outgrows it once A_1-boundedness fails, so the substitution alphabet can no longer be read off as "value of A_n mod κ." No infinite subfamily of the rusty numbers, or of any other family, is closed by this note; whether *every* rusty number d works remains exactly as open as [3, §6] leaves it.

Guessing, not verification, is the practical bottleneck as τ grows: the prefixes needed to find a correct candidate ranged up to 7×10^7 terms ($a = (1, 12)$), located by the engine's counterexample rather than by blind escalation; verifying a found candidate costs seconds to under a minute. The sweep covers $k = 2$ up to $\tau \approx 13$, $k = 3$ to $\tau = 11$, and $k = 4$ to $\tau = 9$; larger cases were not attempted, so 249 is a computational budget boundary, not a mathematical one. Finally, every claim in this note rests on Peanut being a correct decision procedure for $\langle \mathbb{N}, +, V_\tau \rangle$ plus a DFAO; that correctness is not formally verified, though it is supported here by hundreds of adversarial perturbation and random-automaton controls (all correctly rejected), by exact agreement with an independently written re-implementation on all 249 records, and by two independently written sequence generators agreeing with it everywhere tested.

Acknowledgement and declaration of AI usage. This note, and the work behind it, were produced with substantial use of a large language model. This note carries this declaration from its first version. Precisely: the models were Claude (Anthropic): `claude-fable-5` as the orchestrating model, with `claude-opus-5` and `claude-sonnet-5` sub-agents for search, implementation, refereeing and drafting, all used through the Claude Code agent framework; they were used for (i) the literature and prior-art search, including retrieving and checking the exact published wording of Conjecture 2, Definition 1, and Theorem 4 against the journal text; (ii) the design and implementation of Peanut, the open-source decision procedure with which every computation here was run; (iii) the encoding of §3, the guess-and-verify search for each of the 249 DFAOs, and the identification of Observations 6.1–6.2; (iv) an adversarial "referee" pass, also carried out by the model, which re-derived the induction of §3 by hand, found the inverted `LAST` predicate described there, wrote independent brute-force checks and a second obligation-generating script sharing no code with the original, and re-ran all 249 records; and (v) the drafting of this text, most of which is the model's output lightly edited by the author. The author directed the work, chose the target conjecture, read and takes responsibility for every statement, and ran the computations. All finite claims were verified by the independent implementations described in Section 5; the referee transcript and code are public in the Peanut repository. Any errors are the author's.

References

[1] V. Bruyère, G. Hansel, C. Michaux, and R. Villemaire, *Logic and p-recognizable sets of integers*, Bull. Belg. Math. Soc. Simon Stevin **1** (1994), 191–238.
[2] W. Bosma, R. Bruin, R. Fokkink, J. Grube, A. Reuijl, and T. Tromp, *Using Walnut to solve problems from the OEIS*, J. Integer Seq. **28** (2025), Article 25.3.8.
[3] R. Fokkink and G. Joshi, *Anti-recurrence sequences*, Integers **26** (2026), #A24; also arXiv:2506.13337 [math.NT], 16 June 2025.
[4] C. Kimberling and P. J. C. Moses, *Linear complementary equations and systems*, Fibonacci Quart. **57** (2019), no. 5, 97–111.
[5] T. Zaslavsky, *Anti-Fibonacci numbers, a formula*, unpublished manuscript, https://oeis.org/A075326/a075326_2.pdf.

Email address: hingstonandrew@icloud.com